

# Maximal Semantic Clique Enumeration in Vector-Attributed Graphs

Tianyi Gao

Beijing Institute of Technology  
Beijing, China  
tygao\_bit@163.com

Hongchao Qin

Beijing Institute of Technology  
Beijing, China  
qhc\_neu@gmail.com

## ABSTRACT

Maximal clique enumeration (MCE) is a fundamental primitive for discovering cohesive structures in graph data, but structural completeness alone cannot distinguish semantically coherent groups in vector-attributed graphs. Enforcing collective semantic agreement is challenging because a global average-similarity constraint is non-monotone: a feasible clique may have infeasible subsets, invalidating the pivot invariants that make classical MCE scalable. We formalize *average-constrained maximal semantic clique* (ACMSC) enumeration and first establish SEMBK, a correct pivot-free baseline. We then develop two exact algorithms. STRSUB decouples structural and semantic search, applying Tomita pivoting in a  $\tau$ -agnostic structural phase and admissible top-edge and vertex-contribution bounds during exact subset enumeration. MONOSEMMCE specializes in selective thresholds. Its key property is that every feasible clique has a feasible parent obtained by deterministically deleting a minimum-contribution vertex. The resulting reverse search follows only unique-parent edges and generates each feasible state once without visited tables or materialized size layers. Experiments verify exact output equality on all tested configurations. Across eight datasets, STRSUB and MONOSEMMCE jointly complete all 24 common-grid workloads, whereas SEMBK reaches the 1,200-second limit on 15; at  $q_{99}$ , MONOSEMMCE is  $2.17\times$ – $6.24\times$  faster than STRSUB across all eight datasets and reaches  $5.62\times$  parallel speedup. Output studies further show that ACMSCs attain higher semantic purity and vertex coverage than pairwise-threshold mining across  $q_{50}$ – $q_{99}$ .

## PVLDB Reference Format:

Tianyi Gao and Hongchao Qin. Maximal Semantic Clique Enumeration in Vector-Attributed Graphs. PVLDB, 20(1): XXX-XXX, 2027.  
doi:XX.XX/XXX.XX

## 1 INTRODUCTION

Graphs with rich vertex attributes are ubiquitous in modern data management and analytics. Social networks associate users with profile embeddings, knowledge graphs attach entity descriptions to nodes, and biological networks annotate proteins with functional vectors. In all these settings, the attribute space carries semantic meaning that is largely orthogonal to the topological structure: two vertices may be structurally adjacent yet semantically distant,

and vice versa. Identifying cohesive subgraphs that are simultaneously structurally dense and semantically coherent is therefore a recurring need across domains.

The maximal clique is a classic structural model of cohesion: every pair of vertices is connected, guaranteeing the highest possible edge density. However, in vector-attributed graphs, a clique may group vertices that are heterogeneous in the attribute space. Conversely, attribute-based community detection methods (e.g., modularity optimization with attribute homogeneity) often sacrifice structural guarantees and produce loosely connected groups. This tension motivates a joint model that enforces both structural completeness and semantic coherence.

**The ACMSC Problem.** We study the problem of enumerating *average-constrained maximal semantic cliques* (ACMSCs) in a vector-attributed graph. Each vertex carries a real-valued attribute vector; pairwise semantic similarity is measured by cosine similarity. Given a threshold  $\tau$ , a clique  $C$  is *average-constrained* if its average pairwise similarity is at least  $\tau$ . An ACMSC is inclusion-wise maximal: no strict superset of  $C$  is both a clique and average-constrained. The task is to enumerate *all* ACMSCs of the input graph.

This formulation has two salient features. First, the average constraint is a *global* property of the clique: it depends on all  $\binom{|C|}{2}$  pairwise similarities, not on any single edge. This makes it fundamentally different from edge-level semantic constraints (e.g., requiring every edge to exceed a threshold), which are local and monotone. Second, the average is non-monotone over the subset lattice: adding a vertex can raise or lower the average, so a partial clique that currently violates the constraint may still grow into a valid ACMSC, while one that satisfies it may admit no feasible extension. This non-monotonicity invalidates the structural invariants that underpin the efficiency of classical maximal clique enumeration (MCE) algorithms.

**Why Classical MCE Does Not Suffice.** The Bron-Kerbosch (BK) algorithm and its modern variants achieve state-of-the-art performance for MCE through two key techniques: *pivot-based pruning* (Tomita et al.) and *degeneracy ordering* (Eppstein et al.). Pivot pruning relies on the fact that structural maximality is closed under supersets: if a vertex  $u$  is adjacent to the current pivot  $p$ , any maximal clique containing  $u$  also contains  $p$ , so  $u$  can be safely suppressed as an expansion root. Under the average constraint, this closure breaks: a clique containing  $u$  but not  $p$  may satisfy  $\text{avgSim} \geq \tau$  while  $\{p, u\}$  does not. Consequently, pivoting is not merely ineffective—it is *incorrect* for ACMSC enumeration. A direct BK adaptation must therefore forgo pivoting entirely and defer all semantic evaluation to the leaves, degenerating to exhaustive structural enumeration regardless of  $\tau$ .

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.  
doi:XX.XX/XXX.XX

**Our Contributions.** This work formalizes ACMSC enumeration under inclusion-wise maximality and establishes its computational hardness. It develops STRSUB, which separates structural MCE from semantic subset enumeration, and MONOSEMMCE, a canonical reverse search specialized for selective thresholds. Experiments verify exact output equality, characterize their complementary runtime and memory regimes, and measure the resulting pattern space over a fixed threshold grid.

**Paper Organization.** Section 2 formalizes the problem and establishes hardness. Section 3 presents the baseline SEMBK algorithm and explains why pivoting fails. Section 4 develops STRSUB and MONOSEMMCE. Section 5 describes shared implementation optimizations. Section 6 reports experimental results. Section 7 discusses related work, and Section 8 concludes.

## 2 PRELIMINARIES

### 2.1 Notations and Problem Definition

We study a *vector-attributed graph*  $\mathcal{G} = (V, E, \mathbf{X})$ , where  $V$  is the vertex set,  $E \subseteq \binom{V}{2}$  is the edge set, and  $\mathbf{X} = \{\mathbf{x}_v \in \mathbb{R}^d \mid v \in V\}$  assigns a  $d$ -dimensional attribute vector to each vertex.

**DEFINITION 1 (SIMILARITY FUNCTION).** A similarity function  $\text{sim} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow [-1, 1]$  is a symmetric, bounded function measuring the semantic closeness of two attribute vectors. We assume all attribute vectors are  $\ell_2$ -normalized, so that  $\text{sim}$  coincides with cosine similarity and its range is naturally  $[-1, 1]$ .

Throughout this paper we work with normalized vectors and precompute  $\text{sim}(u, v)$  for all  $(u, v) \in E$ . The threshold  $\tau \in [-1, 1]$  is drawn from the same range.

**DEFINITION 2 (CLIQUE).** A set  $C \subseteq V$  is a clique in  $\mathcal{G}$  if every pair of distinct vertices in  $C$  is connected by an edge, i.e.,  $\{u, v\} \in E$  for all  $u \neq v \in C$ .

Classical maximal clique enumeration (MCE) seeks cliques that cannot be extended by any additional vertex while preserving the clique property. In vector-attributed graphs, however, structural density alone is insufficient to characterize semantically coherent groups: a clique may consist of vertices that are structurally adjacent yet semantically heterogeneous. To capture both aspects jointly, we introduce two companion quantities.

**DEFINITION 3 (PAIRWISE SIMILARITY SUM AND AVERAGE PAIRWISE SIMILARITY).** For a clique  $C$  of size  $k = |C| \geq 2$ , the pairwise similarity sum is

$$\text{simSum}(C) = \sum_{\{u, v\} \subseteq C} \text{sim}(u, v),$$

and the average pairwise similarity is

$$\text{avgSim}(C) = \frac{\text{simSum}(C)}{\binom{k}{2}}.$$

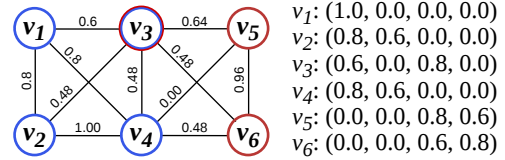
Given a threshold  $\tau \in [-1, 1]$ , we say  $C$  is *average-constrained* if  $\text{avgSim}(C) \geq \tau$ . For algorithmic purposes, we equivalently track the *semantic surplus*

$$W(C) = \text{simSum}(C) - \tau \binom{k}{2},$$

which is non-negative if and only if  $\text{avgSim}(C) \geq \tau$ . Working with  $\text{simSum}$  and  $W$  avoids division at every recursive step;  $\text{avgSim}$  is used only when a human-readable quantity is required.

**DEFINITION 4 (ACMSC).** Given  $\mathcal{G} = (V, E, \mathbf{X})$  and threshold  $\tau \in [-1, 1]$ , a clique  $C \subseteq V$  is an *average-constrained maximal semantic clique (ACMSC)* if (i)  $\text{avgSim}(C) \geq \tau$ , and (ii) there is no clique  $C' \supsetneq C$  such that  $\text{avgSim}(C') \geq \tau$ .

This is inclusion-wise maximality, not merely the absence of a feasible single-vertex extension. The distinction matters because average feasibility is non-monotone. For example, let  $\tau = 0.6$  and let four vertices form a clique with  $\text{sim}(a, b) = 0.6$ ,  $\text{sim}(a, u) = \text{sim}(b, u) = \text{sim}(a, v) = \text{sim}(b, v) = 0.5$ , and  $\text{sim}(u, v) = 1$ . The feasible clique  $\{a, b\}$  has no feasible single-vertex extension, since either three-vertex average is  $1.6/3 < 0.6$ , but the strict superset  $\{a, b, u, v\}$  is feasible with average  $3.6/6 = 0.6$ . Consequently, algorithms may use the absence of a feasible single extension only to generate candidates; a global containment test is still required for ACMSC output.



**Figure 1: Example of vector-attributed graph with semantic similarities.** Each vertex has a 4-dimensional vector, and edge labels show pairwise similarities.

**EXAMPLE 1.** Figure 1 shows a vector-attributed graph with six vertices. Edge labels give the precomputed pairwise similarities. With threshold  $\tau = 0.6$ , consider the two structural cliques

$$M_1 = \{v_1, v_2, v_3, v_4\}, \quad M_2 = \{v_3, v_4, v_5, v_6\}.$$

Let  $C_1 = \{v_1, v_2, v_3, v_4\}$ ,  $C_2 = \{v_3, v_5, v_6\}$ , and  $C_3 = \{v_3, v_4, v_5, v_6\}$ . Their total and average similarities are

$$\begin{aligned} \text{simSum}(C_1) &= 4.16, & \text{avgSim}(C_1) &\approx 0.693 \geq \tau, \\ \text{simSum}(C_2) &= 2.08, & \text{avgSim}(C_2) &\approx 0.693 \geq \tau, \\ \text{simSum}(C_3) &= 2.56, & \text{avgSim}(C_3) &\approx 0.427 < \tau. \end{aligned}$$

Thus  $C_1$  and  $C_2$  are average-constrained, whereas the structurally valid clique  $C_3$  is not.

Moreover, neither  $C_1$  nor  $C_2$  has a strict superset that is both a clique and average-constrained, so both are ACMSCs.

### Problem Statement.

**PROBLEM 1 (ACMSC ENUMERATION).** Given a vector-attributed graph  $\mathcal{G} = (V, E, \mathbf{X})$  and a threshold  $\tau \in [-1, 1]$ , enumerate all ACMSCs of  $\mathcal{G}$ .

## 2.2 Challenges

**Hardness.** ACMSC enumeration generalizes classical MCE: when  $\tau$  is at most the minimum pairwise similarity over all edges, every structural maximal clique satisfies the average constraint, so ACMSC enumeration coincides with MCE and the  $\Omega(3^{n/3})$  lower bound carries over. The associated decision problem is NP-complete.

**THEOREM 2.1.** *Given  $\mathcal{G} = (V, E, \mathbf{X})$ , threshold  $\tau \in [-1, 1]$ , and integer  $k$ , deciding whether an ACMSC of size  $\geq k$  exists is NP-complete.*

PROOF. *NP membership.* The decision instance is positive if and only if it contains an average-constrained clique  $C$  of size at least  $k$ . The forward implication is immediate. For the reverse implication, the finite family of feasible clique supersets of  $C$  contains an inclusion-wise maximal member, whose size is at least  $|C| \geq k$ . A certificate therefore need only provide such a feasible clique  $C$ : verifying all edges and  $\text{avgSim}(C) \geq \tau$  takes  $O(|C|^2)$  time. It is unnecessary, and under the non-monotone constraint insufficient, to verify maximality by checking only single-vertex extensions.

*NP-hardness.* We reduce from the CLIQUE decision problem [24], which asks whether  $G_0 = (V, E)$  contains a clique of size  $\geq k$ . Assign  $\mathbf{x}_v = \mathbf{e}_1$  to every vertex, so  $\text{sim}(u, v) = 1$  for every edge, and set  $\tau = 1$ . Under this assignment  $\text{avgSim}(C) = 1 \geq \tau$  for every clique  $C$ , so every structural maximal clique is an ACMSC and vice versa. Any clique of size  $\geq k$  in  $G_0$  is contained in some structural maximal clique of size  $\geq k$ , which is also an ACMSC of size  $\geq k$ . Conversely, any ACMSC of size  $\geq k$  is a clique of size  $\geq k$  in  $G_0$ . Hence a size- $k$  ACMSC exists if and only if  $G_0$  contains a clique of size  $\geq k$ , completing the reduction.  $\square$

The output sets of ACMSC enumeration and MCE are incomparable: a structural maximal clique with  $\text{avgSim}(M) < \tau$  may contain semantically maximal subsets that MCE never generates, while valid ACMSCs need not be structurally maximal. The output size, and hence the computational cost, is  $\tau$ -dependent in a non-trivial way that we characterize below.

**Output Distribution and Practical Scope.** The number of ACM-SCs and their sizes vary substantially with  $\tau$ . At sufficiently low  $\tau$ , almost every clique satisfies the average constraint and the output converges toward that of classical MCE. As  $\tau$  increases, large structural cliques that previously satisfied the constraint cease to do so and are replaced by many smaller semantically feasible sub-cliques, none of which subsumes another. This *fragmentation effect* can inflate the output size by orders of magnitude relative to MCE. At sufficiently high  $\tau$ , the constraint eliminates most cliques outright and the output shrinks considerably.

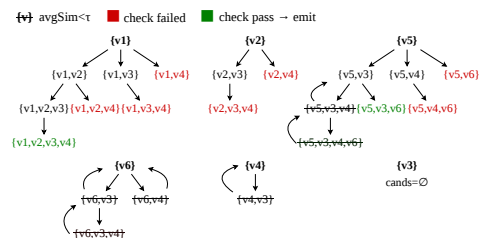
The intermediate fragmentation band carries the largest output volume and is therefore inherently time-consuming—any correct algorithm must enumerate all output cliques, so the output size itself imposes a lower bound on runtime. Low  $\tau$  is appropriate when the goal is to filter out structurally maximal cliques that are semantically too loose; high  $\tau$  is appropriate when the goal is to surface only the most semantically coherent groups. The exact  $\tau$  values at which these transitions occur depend on the similarity distribution of the underlying dataset and cannot be fixed a priori. Section 6 therefore

evaluates a fixed grid of per-dataset edge-similarity quantiles rather than choosing absolute thresholds after observing runtimes.

**Non-monotonicity and Framework Incompatibility.** The core difficulty is that  $\text{avgSim}$  is non-monotone over the subset lattice. In classical MCE, structural feasibility is monotone and maximality is enforced implicitly by the  $P = \emptyset$  invariant at every leaf, so the set of feasible extensions can be characterized solely by the current state of  $C$ . Under the average constraint, no such invariant exists: a partial clique with  $\text{avgSim}(C) < \tau$  may still grow into a valid ACMSC, while one with  $\text{avgSim}(C) \geq \tau$  may admit no feasible extension. Furthermore, whether a candidate vertex can participate in a valid extension depends on its accumulated similarity to the current partial clique, which evolves throughout the recursion and cannot be captured by structural conditions alone. As a result, the pruning mechanisms based on structural monotonicity lose their foundation, and the pivot strategy no longer applies. Direct adaptations of BK are therefore forced to defer all semantic evaluation to the leaves, where  $\tau$  appears only in validation rather than guiding the search itself.

### 3 FRAMEWORK

**Framework Overview.** We derive a baseline search procedure by adapting the Bron–Kerbosch (BK) recursion to the average-constrained semantic clique setting. The standard partition  $(P, X)$  remains useful for structural enumeration, but the semantic constraint is defined at the clique level: whether a partial clique can eventually satisfy the threshold depends on  $\text{avgSim}$  over all vertex pairs within the clique. As a consequence, the pivoting strategy, which relies on local structural properties, is no longer directly applicable. The baseline therefore retains the BK recursion and degeneracy ordering while excluding pivoting, and enforces semantic maximality via a secondary `CHECK` pass on  $X$  at each leaf.



**Figure 2: Search tree of Algorithm 1 on the example graph in Section 2. The traversal follows the degeneracy ordering, and each subtree corresponds to a recursive expansion in Algorithm 1.**

**[Lines 4–8]: Outer Traversal.** The outer loop iterates over vertices in degeneracy order  $\sigma$ . For a seed vertex  $v$ , its later neighbors under  $\sigma$  form the candidate set  $P$ , and its earlier neighbors are placed in  $X$ . This partition guarantees that every structural maximal clique is generated by exactly one seed, because the unique vertex of minimum degeneracy index in any clique serves as its seed and sees all co-members in its  $P$ . Since every ACMSC is contained in at least one structural maximal clique (as established later in Lemma 4.1),

---

**Algorithm 1: Semantic BK with Check**

---

```
1 Input:  $G = (V, E)$ , semantic vectors  $\{x_v\}$ , threshold  $\tau$ 
2 Output: All maximal semantic cliques  $C$  s.t.  $\text{avgSim}(C) \geq \tau$ 
3 Precompute  $\text{sim}(u, v)$  and degeneracy ordering  $\sigma$ ;
4 for each  $v \in \sigma$  in parallel do
5    $P \leftarrow \{u \in N(v) \mid \sigma(u) > \sigma(v)\}$ ;
6    $X \leftarrow \{u \in N(v) \mid \sigma(u) < \sigma(v)\}$ ;
7   Initialize  $\text{acc}[u] \leftarrow \text{sim}(u, v)$  for all  $u \in N(v)$ ;
8   Search( $C = \{v\}, \text{simSum} = 0, P, X, \text{acc}, \text{mode} = \emptyset, 0$ )

9 Function Search( $C, \text{simSum}, P, X, \text{acc}, \text{mode}, k_{\min}$ ):
10   $k \leftarrow |C|$ ,  $\text{foundLonger} \leftarrow \text{false}$ ;
11  for each  $v \in P$  do
12     $\text{simSum}' \leftarrow \text{simSum} + \text{acc}[v]$ ;
13     $P \leftarrow P \setminus \{v\}, P' \leftarrow P \cap N(v), X' \leftarrow X \cap N(v)$ ;
14     $\text{acc}[u] \leftarrow \text{acc}[u] + \text{sim}(u, v)$  for  $u \in P' \cup X'$ ;
15     $\text{res} \leftarrow \text{Search}(C \cup \{v\}, \text{simSum}', P', X', \text{acc}, \text{mode}, k_{\min})$ ;
16     $\text{acc}[u] \leftarrow \text{acc}[u] - \text{sim}(u, v)$  for  $u \in P' \cup X'$ ;
17    if  $\text{res}$  then
18       $\text{foundLonger} \leftarrow \text{true}$ ;
19      if  $\text{mode} = \text{CHECK}$  then return true;
20     $X \leftarrow X \cup \{v\}$ ;
21  if  $\neg \text{foundLonger}$  then
22    if  $k \geq 2 \wedge 2 \cdot \text{simSum} / (k(k-1)) < \tau$  then return false;
23    if  $\text{mode} = \text{CHECK}$  then return  $k > k_{\min}$ ;
24    if  $X \neq \emptyset$  then
25      if Search( $C, \text{simSum}, X, \emptyset, \text{acc}, \text{CHECK}, k$ ) then
26        return true;
27    output  $C$ ;
28    return true;
29  return foundLonger;
```

---

the same partition ensures completeness for the semantic problem without modification.

**[Lines 9–29]: Recursive Search.** Within each recursive call, the algorithm explores all vertices in  $P$  as expansion roots. For each candidate  $v$ , it updates  $\text{simSum}$  and the auxiliary accumulation array  $\text{acc}$ , then recurses on the reduced candidate and exclusion sets ( $P', X'$ ). When the recursion reaches a leaf ( $\text{foundLonger} = \text{false}$ ), it first tests the average constraint: if  $\text{avgSim}(C) < \tau$ , the branch is pruned immediately. Otherwise, semantic maximality must be verified: a clique  $C$  with  $\text{avgSim}(C) \geq \tau$  and  $P = \emptyset$  could still be non-maximal if some vertex in  $X$  can extend it to a larger valid clique. The algorithm therefore invokes SEARCH on  $X$  in CHECK mode with  $k_{\min} = |C|$ . In CHECK mode, the recursion explores extensions through  $X$  and returns *true* as soon as it finds any valid clique strictly larger than  $C$ ; if no such extension exists,  $C$  is emitted. The strict size condition  $k > k_{\min}$  is necessary because the CHECK recursion may reach sub-cliques of  $C$  itself; without recording  $k_{\min}$ , these would be misidentified as valid extensions.

**Why pivoting is not used.** In standard BK, a pivot  $p \in P \cup X$  is selected so that every maximal clique in the current branch contains either  $p$  or a vertex in  $P \setminus N(p)$ . Vertices in  $P \cap N(p)$  are therefore never used as expansion roots: any maximal clique containing such

a vertex  $u$  must also contain  $p$  and will be discovered in the sub-tree rooted at  $p$ . This argument is purely structural and relies on the fact that structural maximality is closed under supersets.

The semantic constraint breaks this closure. Consider  $C = \emptyset$ ,  $P = \{p, u\}$  with  $\{p, u\} \in E$ , and suppose pivot  $p$  is chosen so that  $u$  is suppressed as an expansion root. The only sub-tree explored is rooted at  $p$ . If  $\text{avgSim}(\{p, u\}) < \tau$ , this branch is pruned and yields nothing. If  $\{u, w\}$  for some  $w \in X$  is a valid ACMSC, it is permanently lost: the sub-tree rooted at  $u$  was never entered, and the CHECK pass offers no remedy since it operates only on cliques already constructed, not on suppressed sub-trees. The absence of pivoting is therefore a correctness requirement, not a design choice.

**Complexity.**

**THEOREM 3.1 (TIME AND SPACE COMPLEXITY).** *Let  $\mathcal{L}$  be the feasible terminal states reached by the outer search and let  $X_C$  be the candidate set examined by CHECK for  $C \in \mathcal{L}$ . The running time is  $O^*(n^{2^d} + \sum_{C \in \mathcal{L}} 2^{|X_C|})$ , where  $O^*$  suppresses polynomial set-intersection costs. A graph-independent upper bound is  $O^*(3^n)$ . With  $p$  workers, the implementation uses  $O(m + nD + pn^2)$  worst-case space, where  $D$  is the vector dimension.*

**PROOF.** A degeneracy seed has at most  $d$  later neighbors. Because no pivot is applied, its outer recursion may visit every subset of those neighbors, giving  $O^*(n^{2^d})$  states. For a feasible terminal  $C$ , CHECK may independently visit every subset of  $X_C$ . These states are not subsumed by the outer-tree bound because the same earlier extension can be reconsidered for several terminals. Charging a state to a pair of nested vertex sets yields the safe  $O^*(3^n)$  global bound. Similarities and sparse adjacency require  $O(m + nD)$  storage. Each worker owns an  $O(n)$  accumulator, while the decreasing candidate arrays retained along a worst-case CHECK stack sum to  $O(n^2)$ .  $\square$

**Design Implication.** SEMBK is the direct semantic-search baseline and establishes exact inclusion-wise maximality through its CHECK pass. Its pivot-free branching and repeated extension checks arise from the same property: average similarity couples all pairs in a clique and cannot be summarized by local adjacency alone. Section 4 develops two exact search organizations that expose additional structural or semantic pruning.

## 4 PROPOSED ALGORITHMS

This section presents two exact algorithms that exploit the global structure of the average constraint from different directions.

STRSUB (Section 4.1) resolves the pivot incompatibility by decoupling structural and semantic search. The pivot is applied without modification to a  $\tau$ -agnostic structural phase, restoring the  $O(3^{n/3})$  BK guarantee;  $\tau$  is then enforced in a subsequent intra-clique enumeration phase. Complementary sorted-edge and vertex-contribution upper bounds introduced in Phase 2 skip subset-size layers that cannot satisfy  $\tau$ . Its cost is governed by structural MCE and by the sizes of the resulting host cliques.

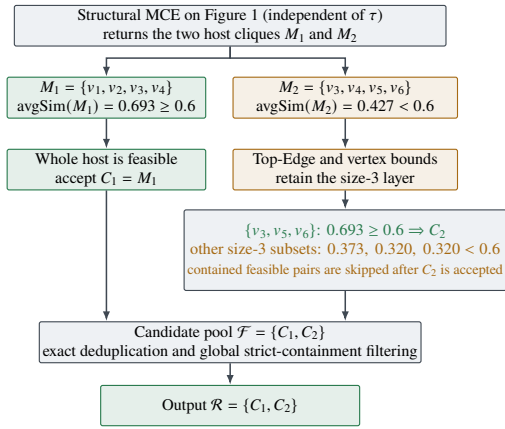
MONOSEMMCE (Section 4.2) departs from the BK framework entirely. Motivated by the observation that STRSUB, despite its pruning improvements, still processes all structural maximal cliques irrespective of their semantic quality, MONOSEMMCE introduces a

canonical reverse-search strategy in which  $\tau$  drives enumeration from the first expansion step. A deterministic feasible parent gives each feasible clique one generation path, while depth-first traversal avoids storing a complete size layer. The method is particularly effective at high  $\tau$ , where few candidate extensions pass the surplus test.

Graph size and structural clique growth determine the crossover between the two methods. We therefore evaluate them on the same quantile grid: STRSUB provides broad threshold coverage, while MONOSEMCE specializes the search for selective high- $\tau$  workloads.

#### 4.1 STRSUB: Structural Decomposition and Subset Enumeration

**Structural–Semantic Decoupling and Containment.** The validity of the Bron–Kerbosch pivot rests on a structural invariant: every structural maximal clique must contain at least one vertex from  $P \setminus N(u)$  for any pivot  $u \in P \cup X$ . This invariant is violated when the average-similarity constraint is incorporated directly into the branching procedure, because  $\text{avgSim}$  is a *global* property of a candidate set and does not decompose along the local neighborhood structure used to justify the pivot. Consequently, the semantic variant SEMBK must forgo pivoting entirely, which undermines the  $O(3^{n/3})$  worst-case guarantee of the standard algorithm [44].



**Figure 3: STRSUB execution after structural MCE on Figure 1.** Gray denotes structural or global processing, amber a failed host or bounded search, and green accepted feasible states. The feasible host  $M_1$  is accepted directly; bounded enumeration inside  $M_2$  recovers  $C_2$ .

The key insight behind STRSUB is that the structural and semantic layers of the problem can be *decoupled*. Specifically, the Tomita pivot can be applied without modification to the purely structural problem of enumerating maximal cliques in  $G$ , because that problem carries no average-similarity constraint. The threshold  $\tau$  is then enforced in a second, independent pass confined to each structural maximal clique. The theoretical basis for this decoupling is the following containment lemma.

---

#### Algorithm 2: STRSUB: Structural Decomposition and Subset Enumeration

---

**Input:**  $G = (V, E)$ , similarities  $\{\text{sim}(u, v)\}_{(u,v) \in E}$ , threshold  $\tau$   
**Output:** All ACMSCs  $\mathcal{R}$

```

1  $\mathcal{F} \leftarrow \emptyset$ ;
  /* Pipelined structural MCE and semantic processing */
2 Build degeneracy ordering  $\sigma$ ;
3 for each  $v \in \sigma$  in parallel do
4    $P \leftarrow \{u \in N(v) \mid \sigma(u) > \sigma(v)\}$ ;
5    $X \leftarrow \{u \in N(v) \mid \sigma(u) < \sigma(v)\}$ ;
6   Run BK( $\{v\}, P, X$ ) with the following callback at each leaf  $M$ ;
  /* Callback: intra-clique semantic enumeration */
7 for each structural leaf  $M$  produced by a worker do
8   if  $\text{avgSim}(M) \geq \tau$  then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{M\}$ ; continue;
  /* Admissible layer bounds */
9   Sort edges of  $M$  by similarity descending;
10   $k^* \leftarrow \max\{k \in [2, |M| - 1] \mid e_k \geq \tau\}$ ;
11  if  $k^*$  does not exist then continue;
12   $lmax \leftarrow \emptyset$ ;  $umask \leftarrow 0$ ;
13  for  $k \leftarrow k^*$  to 2 do
14    Compute the vertex-contribution bound  $\bar{q}_k$  for size  $k$ ;
15    if  $\bar{q}_k < \tau$  then continue;
16    for each size- $k$  subset  $S \subseteq M$  do
17      if  $(S \& umask) = S$  and  $\exists T \in lmax : S \subseteq T$  then
18        continue;
19      if  $\text{avgSim}(S) \geq \tau$  then
20         $lmax \leftarrow lmax \cup \{S\}$ ;
21         $umask \leftarrow umask \mid S$ ;
22         $\mathcal{F} \leftarrow \mathcal{F} \cup \{S\}$ ;
  /* Phase 3: Exact deduplication and strict-containment filtering */
23 Sort  $\mathcal{F}$  by  $|\cdot|$  descending; remove exact duplicates;
24 Initialize an empty inverted index  $inv : v \mapsto \emptyset$ ;
25  $\mathcal{R} \leftarrow \emptyset$ ;
26 for each equal-size group  $\mathcal{G} \subseteq \mathcal{F}$  in descending size do
27   for each  $S \in \mathcal{G}$  in parallel do
28      $r \leftarrow \arg \min_{v \in S} |inv[v]|$ ;
29     Keep  $S$  iff no  $T$  referenced by  $inv[r]$  satisfies  $S \subsetneq T$ ;
30   Insert every kept  $S \in \mathcal{G}$  into  $\mathcal{R}$  and  $inv$ ;
31 return  $\mathcal{R}$ ;
  
```

---

**LEMMA 4.1 (STRUCTURAL SUPERSSET OF ACMSCs).** *Every ACMSC is contained in at least one structural maximal clique of  $G$ . That is, for every ACMSC  $C$ , there exists a structural maximal clique  $M$  such that  $C \subseteq M$ .*

**PROOF.** By definition, every ACMSC is a clique of  $G$ . Let  $C$  be an arbitrary ACMSC, and consider the family

$$\mathcal{F} = \{D \subseteq V \mid C \subseteq D \text{ and } D \text{ is a clique in } G\}.$$

Since  $C \in \mathcal{F}$ , the family is nonempty. Because  $V$  is finite,  $\mathcal{F}$  is finite, so we may choose a set  $M \in \mathcal{F}$  that is maximal under inclusion. By construction,  $M$  is a clique containing  $C$ , and no vertex  $w \in V \setminus M$  extends  $M$  to a larger clique; hence  $M$  is a structural maximal clique of  $G$ , and  $C \subseteq M$ .  $\square$

Lemma 4.1 establishes that the complete collection of ACMSCs can be recovered by (i) enumerating all structural maximal cliques, and (ii) searching within each structural maximal clique for semantically feasible maximal subsets. Step (i) is a purely structural problem to which the full Tomita pivot applies, restoring the  $O(3^{n/3})$  worst-case bound of standard BK. The threshold  $\tau$  enters only in step (ii), where the search space is bounded by the size of a single structural clique. Algorithm 2 implements this strategy in three phases.

**[Lines 1–6]: Phase 1 – Structural Maximal Clique Enumeration.** Phase 1 enumerates all structural maximal cliques of  $G$  using the standard BK algorithm equipped with the Tomita pivot [44] and degeneracy ordering [14]. Vertices are processed in degeneracy order  $\sigma$ ; for each seed vertex  $v$ , the candidate set  $P$  and exclusion set  $X$  are initialized to the forward and backward neighborhoods of  $v$  in  $\sigma$ , respectively, and recursive BK is invoked on the triple  $(\{v\}, P, X)$ . Seeds are dispatched in parallel via OpenMP dynamic scheduling. Crucially, this phase is entirely  $\tau$ -agnostic: the threshold plays no role in branching decisions, so the full pivot strategy and its associated complexity bound are preserved. The implementation does not materialize the set of all structural maximal cliques: when BK reaches a leaf  $M$ , the same worker immediately invokes the Phase 2 callback and retains only its thread-local semantic candidates.

**[Lines 7–20]: Phase 2 – Intra-Clique Semantic Enumeration.** For each structural maximal clique  $M$  of size  $m$  produced by Phase 1, Phase 2 enumerates all semantically maximal subsets of  $M$  with respect to  $\tau$ . Subsets of  $M$  are generated in descending order of cardinality using native-word bitmasks when possible and dynamic bitsets for larger host cliques; no mask-width limit is imposed. Pairwise similarities are precomputed into an  $m \times m$  local matrix to amortize repeated avgSim evaluations.

A *local maximality filter* prunes redundant candidates during enumeration. The filter maintains a set  $lmax$  of valid subsets discovered so far within  $M$ , together with a union mask  $umask = \bigcup_{T \in lmax} T$  that provides a constant-time necessary condition for subsumption. A candidate subset  $S$  is skipped if  $S \subseteq T$  for some  $T \in lmax$ , verified first via the bitmask test  $(S \& umask) = S$  and then by a linear scan over  $lmax$ . This filter is correct because, if  $S \subseteq T$  and  $\text{avgSim}(T) \geq \tau$ , then  $S$  cannot be semantically maximal in  $G$  regardless of its own average similarity. Structural leaves are processed by their owning workers. The candidate size layers are further reduced by the admissible bounds described below.

**[Lines 21–28]: Phase 3 – Global Deduplication.** Because the same ACMSC may be structurally contained in multiple distinct maximal cliques, Phase 2 may emit duplicate or subsumed candidates across cliques. Phase 3 consolidates these candidates into the final result set  $\mathcal{R}$ .

The phase proceeds in two steps. First, candidates are sorted by decreasing size with a lexicographic tie break, and exact duplicates are removed in  $O(N \log N)$  time, where  $N = |\mathcal{F}|$ . Second, equal-size groups are processed in that order. The inverted index  $inv : v \mapsto \{T \in \mathcal{R} \mid v \in T\}$  contains only retained strict supersets of the current group. For each candidate  $S$ , the algorithm identifies its *rarest vertex*  $r = \arg \min_{v \in S} |inv[v]|$  and scans only  $inv[r]$ ; a sorted-merge traversal determines in  $O(|S| + |T|)$  time whether  $S \subseteq T$ . Queries within one group read the same frozen index and run in parallel, after which the retained candidates are inserted.

Equal-size sets cannot strictly contain one another. Moreover, if a discarded larger candidate contains  $S$ , transitivity guarantees that a retained candidate containing both will be indexed. The grouped filter is therefore equivalent to testing all larger candidates while avoiding their indexing and synchronization costs.

**Admissible Layer Pruning.** Phase 2 as described iterates over all subset sizes from  $|M| - 1$  down to 2. A key observation allows the starting size to be reduced: the best achievable avgSim for any size- $k$  subset of  $M$  is bounded above by the mean of the  $\binom{k}{2}$  largest pairwise similarities in  $M$ .

**PROPERTY 4.1 (SORTED-EDGE UPPER BOUND).** *Let  $e_1 \geq e_2 \geq \dots \geq e_{\binom{m}{2}}$  be the pairwise similarities of  $M$  sorted in descending order. For any size- $k$  subset  $S \subseteq M$ ,*

$$\text{avgSim}(S) \leq \bar{e}_k := \frac{1}{\binom{k}{2}} \sum_{i=1}^{\binom{k}{2}} e_i.$$

**PROOF.** Any size- $k$  subset  $S \subseteq M$  uses exactly  $\binom{k}{2}$  pairs, each contributing at most the corresponding value in the sorted sequence  $e_1 \geq e_2 \geq \dots$ ; hence  $\text{simSum}(S) \leq \sum_{i=1}^{\binom{k}{2}} e_i$ , and dividing both sides by  $\binom{k}{2}$  gives the result.  $\square$

The sorted-edge bound ignores that its largest edges may be concentrated on too few vertices to coexist in one size- $k$  subset. A complementary bound tracks which vertices support those similarities. For each  $v \in M$ , let  $q_v(k)$  be the sum of the  $k - 1$  largest similarities between  $v$  and vertices in  $M \setminus \{v\}$ . Let  $q_{(1)}(k) \geq \dots \geq q_{(m)}(k)$  denote these values in descending order.

**PROPERTY 4.2 (VERTEX-CONTRIBUTION UPPER BOUND).** *For any size- $k$  subset  $S \subseteq M$ ,*

$$\text{avgSim}(S) \leq \bar{q}_k := \frac{1}{k(k-1)} \sum_{i=1}^k q_{(i)}(k).$$

**PROOF.** For every  $v \in S$ , the sum of its similarities to the other vertices of  $S$  uses  $k - 1$  vertices from  $M \setminus \{v\}$  and is therefore at most  $q_v(k)$ . Summing over  $S$  counts each internal pair twice, so

$$2\text{simSum}(S) \leq \sum_{v \in S} q_v(k) \leq \sum_{i=1}^k q_{(i)}(k).$$

Division by  $k(k-1)$  gives the claimed average-similarity bound.  $\square$

Note that  $\bar{e}_k$  is non-increasing in  $k$ : a larger subset requires more edges, drawing further into the sorted list and lowering the mean. Before enumeration, all pairwise similarities of  $M$  are sorted once in  $O(m^2 \log m)$  and prefix means  $\bar{e}_k$  are computed for all  $k$ . The algorithm identifies

$$k^* = \max\{k \in [2, |M| - 1] \mid \bar{e}_k \geq \tau\},$$

and begins enumeration at  $k = k^*$  rather than  $|M| - 1$ . Because  $\bar{e}_k$  is non-increasing, all sizes  $k > k^*$  satisfy  $\bar{e}_k < \tau$ , certifying that no subset of those sizes can meet the constraint; they are skipped entirely. If no such  $k^*$  exists, the whole clique  $M$  is skipped. For every remaining size  $k$ , the algorithm also skips the layer when  $\bar{q}_k < \tau$ . The two tests are independently admissible, so using their minimum cannot remove a feasible subset. Computing all per-vertex

similarity lists and prefix sums costs  $O(m^2 \log m)$ , matching the sorting order of the Top-Edge preprocessing.

**Characteristics and Applicable Regime.** STRSUB inherits the full efficiency of BK with Tomita pivot in Phase 1, which runs in  $O(dn \cdot 3^{d/3})$  time independently of  $\tau$ . At low  $\tau$ , nearly every subset of a structural maximal clique satisfies the average constraint, so the local maximality filter rapidly identifies large valid subsets and terminates Phase 2 early. The two layer bounds extend this efficiency to medium and high  $\tau$ : for most structural cliques  $M$ , either the whole clique is accepted immediately ( $\text{avgSim}(M) \geq \tau$ ) or  $k^*$  is small, confining bitmask enumeration to a narrow band of sizes near the base. At high selectivity, Phase 1 still enumerates every structural maximal clique. MONOSEMMCE complements this organization by applying  $\tau$  from the first expansion and storing only depth-first search state.

#### Complexity.

**THEOREM 4.1 (TIME COMPLEXITY OF STRSUB).** *STRSUB runs in*

$$O\left(dn \cdot 3^{d/3} + \sum_{M \in \mathcal{M}} 2^{|M|} |M|^2 + KN \log N + K \sum_{S \in \mathcal{F}} L_{\min}(S)\right)$$

*time, where  $d$  is the degeneracy of  $G$ ,  $N = |\mathcal{F}|$  is the number of candidates before deduplication,  $K = \max_{S \in \mathcal{F}} |S|$ , and  $L_{\min}(S) = \min_{v \in S} |\text{inv}[v]|$ .*

**PROOF. Phase 1.** With degeneracy ordering, BK with Tomita pivot runs in  $O(dn \cdot 3^{d/3})$  time [14].

**Phase 2.** For a structural maximal clique  $M$  of size  $m$ , bitmask enumeration visits at most  $2^m$  subsets. Each subset  $S$  requires computing  $\text{avgSim}(S)$  from the precomputed  $m \times m$  similarity matrix at cost  $O(|S|^2) \leq O(m^2)$ . The per-clique cost is therefore  $O(2^m \cdot m^2)$ , giving  $\sum_{M \in \mathcal{M}} O(2^{|M|} \cdot |M|^2)$  overall. The union-mask pre-filter and local maximality scan do not increase this asymptotic bound.

**Phase 3.** Lexicographic sorting costs  $O(KN \log N)$ . For each candidate  $S$ , the algorithm scans  $\text{inv}[r]$  of length  $L_{\min}(S)$ ; each of the at most  $L_{\min}(S)$  subsumption checks costs  $O(K)$  via sorted-merge comparison. The total Phase 3 cost is  $O(KN \log N + K \sum_{S \in \mathcal{F}} L_{\min}(S))$ .  $\square$

**Remark.** The  $\sum_{M \in \mathcal{M}} 2^{|M|} \cdot |M|^2$  term is a worst-case bound on Phase 2 without pruning. In practice, the sorted-edge bound (Property 4.1) restricts enumeration to sizes  $k \leq k^*$ , and the vertex-contribution bound (Property 4.2) may remove additional individual layers, reducing the per-clique cost to  $O(|M|^2 \cdot \sum_{k=2}^{k^*} \binom{|M|}{k})$ , which is substantially smaller than  $2^{|M|} \cdot |M|^2$  whenever  $k^*$  is small relative to  $|M|$ .

## 4.2 MONOSEMMCE: High- $\tau$ Canonical Reverse Search

**Motivation and Core Insight.** Even with the Top-Edge Layer Pruning, STRSUB is bound to Phase 1: it enumerates all structural maximal cliques before any semantic evaluation takes place. In the high- $\tau$  regime, the vast majority of structural cliques may be semantically barren, yet each still contributes Phase 1 cost. More fundamentally, the BK framework offers no mechanism to incorporate  $\tau$  into branching decisions—the threshold appears only in

validation, never in the search logic itself. This structural overhead motivates abandoning the BK skeleton entirely.

The obstacle to integrating  $\tau$  directly into search is the non-monotonicity of  $\text{avgSim}$ : an infeasible clique can have a feasible strict superset. A search that retains only feasible states is therefore complete only if every feasible target has a feasible construction path. Theorem 4.2 supplies such a path through the deletion rule defined next.

For a vertex  $v$  in a clique  $C$ , define its *similarity contribution* as  $a_C(v) = \sum_{u \in C \setminus \{v\}} \text{sim}(u, v)$ : the sum of its similarities to the other vertices in  $C$ . We delete a vertex with minimum similarity contribution, breaking ties by vertex identifier. The theorem below proves that the remaining clique is feasible. This deterministic rule gives every feasible clique a unique parent, called its *canonical parent*. Reverse search accepts an extension only when its parent under this rule is the current clique. Thus every feasible clique is generated once without a visited table or per-layer deduplication.

The implementation traverses each canonical tree depth-first and parallelizes its feasible edge roots. It rejects an extension whenever its updated surplus is negative. Unlike the previous level-synchronous algorithm, it stores no complete size layer; working memory consists of the graph representation, thread-local accumulators, DFS scratch space, and output candidates. A final containment filter is still necessary, because absence of a feasible single extension is weaker than inclusion-wise maximality.

#### The Monotone Path Property.

**THEOREM 4.2 (FEASIBLE MINIMUM-CONTRIBUTION DELETION).** *Let  $C$  be an average-constrained clique of size  $k \geq 3$ , with the similarity contribution  $a_C(v)$  defined above. If  $d(C)$  minimizes the pair  $(a_C(v), v)$  lexicographically, then  $C \setminus \{d(C)\}$  is average-constrained. Repeated application of this rule therefore reaches a feasible edge, and reversing these deletions gives a non-increasing-average insertion path.*

**PROOF.** Each pair is counted once at each endpoint. The minimum contribution therefore satisfies  $a_C(d(C)) \leq 2\text{simSum}(C)/k$ . Deleting  $d(C)$  leaves similarity sum at least  $\text{simSum}(C)(k-2)/k$ , and hence

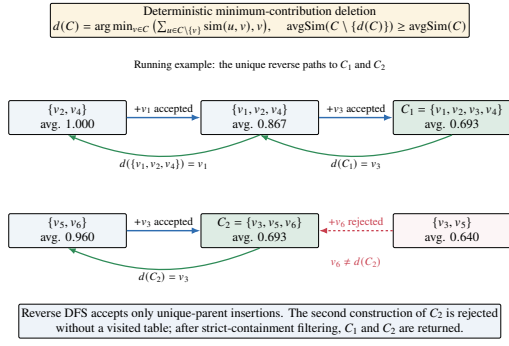
$$\text{avgSim}(C \setminus \{d(C)\}) \geq \frac{\text{simSum}(C)(k-2)/k}{\binom{k-1}{2}} = \text{avgSim}(C) \geq \tau.$$

Deterministic tie-breaking makes  $d(C)$  unique. Applying the same argument until two vertices remain preserves feasibility at every deletion; reversing the sequence proves the insertion-path claim.  $\square$

**Feasible Edge Roots.** Every feasible clique of size at least two reaches a feasible edge under the deterministic deletion rule, so Algorithm 3 starts one tree for each edge with similarity at least  $\tau$ . A state stores the clique  $C$ , its pairwise sum  $S$ , the common-neighbor set  $P$ , candidate accumulators  $\text{acc}$ , and the member contributions  $a_C$ . No BK exclusion set is required: uniqueness follows from the canonical-parent test rather than from an insertion order.

**Surplus and Canonical-Child Tests.** For  $k = |C|$  and  $\delta = \text{acc}[v]$ , the extension satisfies

$$W(C \cup \{v\}) = W(C) + \delta - \tau k.$$



**Figure 4: Canonical reverse search on the running example. Deleting the vertex with minimum similarity contribution never lowers average similarity, yielding one feasible reverse path to each target; the alternative construction of  $C_2$  is rejected by unique-parent ownership.**

---

**Algorithm 3: MONOSEMMCE: Canonical Reverse Search**

---

```

1 Input:  $G = (V, E)$ , semantic vectors  $\{x_v\}$ , threshold  $\tau$ 
2 Output: All ACMSCs  $\mathcal{R}$ 
3 Precompute  $\text{sim}(u, v)$  for all  $(u, v) \in E$ ;
4  $\mathcal{F} \leftarrow \emptyset$ ;
5 for each  $(u, v) \in E$  with  $u < v$  and  $\text{sim}(u, v) \geq \tau$  in parallel do
6    $C \leftarrow \{u, v\}$ ;  $S \leftarrow \text{sim}(u, v)$ ;
7    $P \leftarrow N(u) \cap N(v)$ ;
8    $\text{acc}[w] \leftarrow \text{sim}(w, u) + \text{sim}(w, v)$  for  $w \in P$ ;
9    $a_C(u) = a_C(v) \leftarrow \text{sim}(u, v)$ ;
10  Search( $C, S, P, \text{acc}, a_C$ );
11 Apply Algorithm 2's grouped strict-containment filter to  $\mathcal{F}$ ,
   obtaining  $\mathcal{R}$ ;
12 return  $\mathcal{R}$ ;

13 Function Search( $C, S, P, \text{acc}, a_C$ ):
14    $k \leftarrow |C|$ ;  $W \leftarrow S - \tau \binom{k}{2}$ ;
15   hasValid  $\leftarrow$  false;
16   for each  $v \in P$  do
17      $\delta \leftarrow \text{acc}[v]$ ;
18     if  $W + \delta - \tau k < 0$  then continue;
19     hasValid  $\leftarrow$  true;
20      $D \leftarrow C \cup \{v\}$ ;
21     Update  $a_D$  from  $a_C$  and  $\text{sim}(v, u)$  for  $u \in C$ ;
22     if  $v \neq \arg \min_{x \in D} (a_D(x), x)$  then continue;
23      $P' \leftarrow P \cap N(v)$ ;
24      $\text{acc}'[w] \leftarrow \text{acc}[w] + \text{sim}(w, v)$  for  $w \in P'$ ;
25     Search( $D, S + \delta, P', \text{acc}', a_D$ );
26   if not hasValid then  $\mathcal{F} \leftarrow \mathcal{F} \cup \{C\}$ ;

```

---

If this value is negative, the child itself is infeasible and cannot lie on the canonical path of any feasible answer, so it is safely skipped. For a feasible child  $D = C \cup \{v\}$ , the algorithm updates every member's similarity contribution and accepts  $D$  only if  $v = d(D)$ . Equivalently, the canonical parent of  $D$  must be exactly  $C$ . This test

removes alternative insertion paths before recursion rather than after state materialization.

The path property is used to assign unique parent ownership rather than to order candidates. The algorithm tests child feasibility directly and then applies the minimum-contribution deletion. This removes duplicate paths before recursion without a candidate-order cutoff. Accumulator values are restored from saved values when backtracking, so the result does not depend on reversing a sequence of floating-point additions.

**Inclusion-Wise Maximality.** A state with no feasible single-vertex extension is added to  $\mathcal{F}$ , but is not yet declared an ACMSC. If such a candidate  $C$  is not inclusion-wise maximal, the finite family of feasible clique supersets of  $C$  contains an inclusion-wise maximal member  $M$ . The canonical search enumerates  $M$  exactly once, and  $M$  has no feasible single extension, so it also belongs to  $\mathcal{F}$ . The final grouped inverted-index containment filter consequently removes  $C$ . Conversely, an ACMSC is not contained in any other feasible candidate and is retained.

**THEOREM 4.3 (ENUMERATION CORRECTNESS).** *Algorithm 3 generates every average-constrained clique of size at least two exactly once and returns exactly all ACMSCs.*

**PROOF.** Existence follows by reversing the repeated deterministic deletions in Theorem 4.2; every prefix is feasible and therefore passes the surplus test. Uniqueness follows because every non-root feasible clique has the single parent  $C \setminus \{d(C)\}$ , and only that parent accepts it as a child. The preceding containment argument proves that the final filter retains precisely the inclusion-wise maximal feasible cliques.  $\square$

**Complexity.**

**THEOREM 4.4 (TIME AND SPACE COMPLEXITY OF MONOSEMMCE).** *Let  $F_\tau$  be the number of average-constrained cliques of size at least two,  $K$  the maximum clique size,  $\Delta$  the maximum degree,  $T$  the number of worker threads, and  $N = |\mathcal{F}|$  the number of terminal candidates before containment filtering. A conservative bound for canonical search is*

$$O\left(F_\tau \Delta K \log \Delta + KN \log N + K \sum_{S \in \mathcal{F}} L_{\min}(S)\right),$$

*and its space usage is  $O(m + T(n + K\Delta) + NK)$  words, in addition to vector storage.*

**PROOF.** Canonical uniqueness bounds accepted search states by  $F_\tau$ . At one state, at most  $\Delta$  candidates are tested. Updating and comparing the  $O(K)$  similarity contributions uses cached edge similarities, each found in  $O(\log \Delta)$  time; candidate intersection and accumulator updates fit within the stated conservative per-state bound. The inverted-index filter sorts  $N$  candidates and performs at most  $L_{\min}(S)$  exact  $O(K)$  subset checks for each  $S$ .

Similarity adjacency lists and sparse bitmaps use  $O(m)$  words. Each worker owns an  $O(n)$  generation-tagged accumulator and at most  $O(K\Delta)$  DFS scratch storage. Crucially, this term depends on search depth rather than the number of feasible cliques at any size. The terminal candidates occupy  $O(NK)$  words.  $\square$

**Remark.** The canonical tree contracts as  $\tau$  reduces the feasible-state family  $F_\tau$ , giving MONOSEMMCE its high-threshold operating

region. STRSUB provides the complementary structural decomposition across broader thresholds.

## 5 IMPLEMENTATION OPTIMIZATIONS

The following optimizations are applied to MONOSEMMCE and STRSUB without affecting correctness. They address three categories of bottlenecks: parallel load balance, repeated computation in the expansion inner loop, and memory access cost for neighbor intersection and similarity lookup.

**Parallel Decomposition and Load Balance.** STRSUB Phase 1 parallelizes BK over seed vertices under degeneracy ordering: each seed  $v$  spawns an independent subproblem over its forward neighborhood  $(P_v, X_v)$ , and distinct seeds share no mutable state and require no synchronization. Phase 2 normally processes each structural maximal clique with thread-local candidate lists, but a few large host cliques can dominate this work. For such a clique, each fixed-cardinality layer is partitioned by its smallest selected vertex. These partitions are disjoint and may run concurrently because distinct equal-size subsets cannot strictly contain one another. A barrier merges the feasible subsets before the next smaller layer, preserving all local-containment decisions while exposing parallelism inside a single expensive host clique. Thread-local candidates are merged only after processing, avoiding lock contention on the output.

MONOSEMMCE assigns each feasible edge root to one canonical tree. Distinct trees contain disjoint feasible-clique states, so roots are dispatched with dynamic OpenMP scheduling and require no shared visited table. Each worker performs DFS with thread-local accumulators, scratch buffers, statistics, and candidate output; only the final merge and containment filter synchronize globally.

**Incremental Candidate Maintenance.** When expanding state  $(C, \text{simSum}, P, \text{acc})$  by vertex  $v$ , the child state requires an updated candidate set  $P' = P \cap N(v)$  and an updated accumulator  $\text{acc}'$ . A naïve implementation recomputes  $\text{acc}'$  from scratch at cost  $O(|P'| \cdot |C|)$  per expansion.

Since  $\text{acc}[w] = \sum_{u \in C} \text{sim}(w, u)$ , the child accumulator satisfies

$$\text{acc}'[w] = \text{acc}[w] + \text{sim}(w, v) \quad \forall w \in P',$$

reducing the update to  $O(|P'|)$  regardless of clique depth. The DFS records each previous accumulator value and restores it exactly after the recursive call. Reversing an update by subtracting the same floating-point similarity is not equivalent: rounding can accumulate across sibling branches and change a threshold decision. Candidate ordering has no correctness role in canonical search, so the standard implementation performs no per-state sort.

**Sparse Blocked Bitmap for Neighbor Intersection.** Candidate set intersection  $P' = P \cap N(v)$  is the innermost operation of every expansion. We represent each adjacency list as a *sparse blocked bitmap*: vertices are indexed by  $\lfloor v/64 \rfloor$ , and only non-empty 64-bit blocks are stored as (index, word) pairs in sorted order. Intersection of two rows executes a merge-style scan over their block lists, applying bitwise AND only to matching block indices and extracting set bits via `ctz`. The cost is  $O(b_u + b_v)$  where  $b_u, b_v$  count non-empty blocks in each row, compared to  $O(n/64)$  for a dense bitmap.

This advantage compounds with search depth: at layer  $\ell$ , every candidate  $w \in P$  is a common neighbor of all  $\ell+2$  clique members,

so  $|P|$  decreases with  $\ell$  and each row’s active block count shrinks accordingly. The per-intersection cost therefore decreases with depth.

**Similarity Computation with Reorder-Aware Caching.** All pairwise similarities are precomputed once and stored in sorted adjacency lists for  $O(\log \deg(u))$  lookup via binary search. Inner products use 8-wide AVX2 SIMD over the float32 vector store. Edge similarities remain float32 to control memory traffic, while similarity sums, per-candidate accumulators, surpluses, and threshold products are maintained in double precision. Every feasibility test compares these double-precision quantities directly with an inclusive boundary; no algorithm-specific epsilon is used.

To reduce cache miss cost on repeated lookups, vertices are reordered by a degree-descending BFS before vectors are stored contiguously in memory. A vertex  $u$  with degree  $d_u$  participates in  $O(d_u)$  expansions as a candidate across the search; placing high-degree vertices at low memory offsets concentrates the most frequently accessed vectors into a small number of cache lines, reducing cold misses on the hot working set.

## 6 EXPERIMENTS

### 6.1 Experimental Setup

**Algorithms.** We evaluate four implementations derived from one shared graph and vector backend. STRUCTBK is a Tomita-pivot structural MCE implementation and serves as the single structural reference; it does not solve ACMSC. SEMBK is the pivot-free semantic baseline of Section 3. STRSUB and MONOSEMMCE are the exact algorithms of Section 4. Runtime experiments use instrumentation-free binaries. Search counters use separately compiled statistics modes and never contribute timing values.

**Datasets and attributes.** Table 1 fixes all dataset roles before comparison. The common grid contains three graphs with natural semantic vectors and five public graphs with controlled vectors. FB15K-237 and WN18RR entity descriptions are encoded by all-MiniLM-L6-v2; ogbn-arxiv retains its official 128-dimensional node features [20, 45]. For the public structural graphs from SNAP, SuiteSparse, and the Enron communication collection [11, 26], PCG64 draws 256-dimensional coordinates independently from  $U[-1, 1)$  using a fixed dataset seed, after which the shared loader applies  $\ell_2$  normalization. Thus only attributes are generated; vertices and edges remain those of the public graph. The same graph/vector provenance rule applies to every generated attribute row in the table.

For the scale experiment, vertices of ogbn-arxiv are ordered by decreasing source-graph degree with original identifier as the tie break. Each level is the full induced graph on an order prefix and copies the corresponding official feature rows. The provenance record stores the construction rule, identifier mapping, and source/output checksums. Every level retains all induced edges and the corresponding official feature rows.

**Environment.** Experiments run on an AMD Ryzen 9 7945HX workstation with 16 physical cores, 32 hardware threads, and 32 GiB memory under Windows 11 and MSYS2 GCC 13.2. Every process has a 16 GiB hard memory limit and uses at most 32 OpenMP

| Dataset     | $ V $   | $ E $      | Dim. | Evaluation role        |
|-------------|---------|------------|------|------------------------|
| FB15K-237   | 14,541  | 248,611    | 384  | natural, common grid   |
| WN18RR      | 40,943  | 75,762     | 384  | natural, common grid   |
| ogbn-arxiv  | 169,343 | 1,157,799  | 128  | natural, common grid   |
| email-Enron | 36,692  | 183,831    | 256  | generated, common grid |
| sc-nasasrb  | 54,870  | 1,311,227  | 256  | generated, common grid |
| sc-pkustk11 | 87,804  | 2,565,054  | 256  | generated, common grid |
| sc-pwtk     | 217,891 | 5,653,221  | 256  | generated, common grid |
| sc-ldoor    | 909,537 | 20,770,807 | 256  | generated, common grid |

**Table 1: Public graphs, vector dimensions, and fixed evaluation roles. “Generated” denotes reproducible vectors assigned to unchanged public graph structures.**

threads. Following the common use of censored outcomes in large-graph mining evaluations [28, 36], the formal evaluation harness applies a uniform 1,200-second limit and records timeout and out-of-memory outcomes without a result count. Completed configurations are run three times and reported by median algorithm time; a censored configuration is run once because repetition cannot change its status under the fixed limit.

**Threshold protocol.** Absolute cosine values have different meanings across vector sources. We therefore pre-register per-dataset thresholds as quantiles of cosine similarity over existing graph edges. The common grid uses  $q_{20}$ ,  $q_{80}$ , and  $q_{99}$ ; the sensitivity grid adds  $q_5$ ,  $q_{50}$ ,  $q_{70}$ ,  $q_{90}$ ,  $q_{95}$ , and  $q_{97}$ . This grid includes low, intermediate, transition, and highly selective regimes without choosing a dataset-specific point after observing runtime. All methods receive the same stored float32 edge similarities and use double-precision sums with an inclusive feasibility boundary. On sc-ldoor, MONOSEMMCE reaches the 1,200-second limit at  $q_{20}$ ; lowering  $\tau$  only enlarges its feasible root and canonical-state families, so  $q_5$  is censored as TLE without a redundant execution.

## 6.2 Exactness Before Performance

The vertex-contribution bound matched exact size-layer optima on 225,000 oracle instances and never underestimated an attainable average. Canonical reverse search matched exhaustive feasible-state ownership and inclusion-maximal output on 40,001 oracle instances. The suite covers negative similarities, equality at  $\tau$ , tied vertex contributions, and feasible supersets unreachable through some insertion orders.

Real-data checks compare normalized clique sets and per-size histograms rather than aggregate counts alone. All three semantic methods return the same 577 ACMSCs on WN18RR at  $\tau = 0.8$  and the same 226,936 ACMSCs on ogbn-arxiv at  $q_{80}$ . STRSUB and MONOSEMMCE agree on all 7,476 FB15K-237 outputs at  $q_{99}$  as complete normalized sets. Their aggregate counts and every size-layer count also agree on all jointly completed rows in Table 2, including 13,473,317 sc-ldoor outputs at  $q_{80}$ . Each STRSUB ablation preserves the full 76,106-clique FB15K-237 result. These checks cover the final strict-containment filter required by Definition 4, not only search-tree ownership.

## 6.3 Overall Efficiency

Table 2 is the first end-to-end overview: it applies the same three quantiles to all eight datasets in Table 1, thereby exposing low-, intermediate-, and high-selectivity operating patterns. STRSUB completes 22 of 24 cells and every  $q_{20}$  row, confirming that structural decomposition provides broad low-threshold coverage. At  $q_{80}$ , the ranking depends on graph structure: STRSUB leads on FB15K-237 and ogbn-arxiv, while MONOSEMMCE leads on the other six datasets. At  $q_{99}$  both methods complete every dataset, and canonical search is  $2.17\times$ – $6.24\times$  faster. Thus the high-threshold specialization is visible across natural and generated attributes rather than only on one extreme instance.

SEMBK provides the direct semantic-search reference in the same grid. All three methods complete WN18RR within 0.023–0.066 seconds. On ogbn-arxiv, the fastest specialized method improves over SEMBK by  $1.38\times$ – $6.69\times$ ; on email-Enron, the improvement is  $70.8\times$ – $291.2\times$ . SEMBK reaches TLE throughout the FB15K-237 and generated-attribute rows except email-Enron. STRUCTBK quantifies the structural enumeration cost independently of  $\tau$ ; on ogbn-arxiv it enumerates 1,041,789 structural maximal cliques in 0.309 seconds.

## 6.4 Threshold and Scale Regimes

Figure 5 adds the transition quantiles to two representative regimes. On ogbn-arxiv, STRSUB leads through  $q_{90}$ , whereas MONOSEMMCE becomes  $2.17\times$  faster at  $q_{95}$  and  $4.13\times$  faster at  $q_{97}$ . The crossover therefore appears before the extreme  $q_{99}$  point. The sc-ldoor panel uses the same nine-point grid and shows a wider transition: STRSUB completes the low endpoint, while MONOSEMMCE completes the selective range from  $q_{70}$  through  $q_{99}$ . The two panels show that the crossover location depends on both semantic selectivity and structural workload.

Figure 6 jointly varies threshold and graph size on nested ogbn-arxiv induced subgraphs. Positive cells indicate that STRSUB is faster; negative cells indicate that MONOSEMMCE is faster. The crossover remains at  $q_{95}$  from 25K vertices through the full 169,343-vertex graph. Thus high semantic selectivity, rather than a single favorable graph size, drives the canonical-search advantage. At  $q_{99}$  the full graph takes 0.707 seconds for STRSUB and 0.200 seconds for MONOSEMMCE while both return 10,261 ACMSCs.

| Dataset     | Quantile | ACMSC count | STRUCTBK<br>(structure only) | SEMBK        | STRSUB       | MONOSEMMCE    |
|-------------|----------|-------------|------------------------------|--------------|--------------|---------------|
| FB15K-237   | $q_{20}$ | 206,470     | 0.103                        | TLE          | <b>0.221</b> | TLE           |
|             | $q_{80}$ | 76,106      |                              | TLE          | <b>9.132</b> | TLE           |
|             | $q_{99}$ | 7,476       |                              | TLE          | 45.234       | <b>10.091</b> |
| WN18RR      | $q_{20}$ | 55,200      | 0.015                        | <b>0.037</b> | 0.058        | 0.066         |
|             | $q_{80}$ | 14,379      |                              | <b>0.031</b> | 0.048        | 0.036         |
|             | $q_{99}$ | 750         |                              | 0.035        | 0.050        | <b>0.023</b>  |
| ogbn-arxiv  | $q_{20}$ | 925,239     | 0.309                        | 2.783        | <b>1.233</b> | 3.357         |
|             | $q_{80}$ | 226,936     |                              | 1.206        | <b>0.872</b> | 1.610         |
|             | $q_{99}$ | 10,261      |                              | 1.318        | 0.843        | <b>0.197</b>  |
| email-Enron | $q_{20}$ | 223,703     | 0.090                        | 18.330       | <b>0.259</b> | 16.739        |
|             | $q_{80}$ | 60,352      |                              | 21.369       | 1.266        | <b>0.083</b>  |
|             | $q_{99}$ | 1,819       |                              | 9.608        | 0.206        | <b>0.033</b>  |
| sc-nasasrb  | $q_{20}$ | 28,154      | 0.090                        | TLE          | <b>0.301</b> | TLE           |
|             | $q_{80}$ | 834,914     |                              | TLE          | 102.820      | <b>0.759</b>  |
|             | $q_{99}$ | 12,742      |                              | TLE          | 0.415        | <b>0.079</b>  |
| sc-pkustk11 | $q_{20}$ | 22,612      | 0.143                        | TLE          | <b>0.645</b> | TLE           |
|             | $q_{80}$ | 2,214,411   |                              | TLE          | TLE          | <b>2.219</b>  |
|             | $q_{99}$ | 24,891      |                              | TLE          | 0.611        | <b>0.167</b>  |
| sc-pwtk     | $q_{20}$ | 58,523      | 0.286                        | TLE          | <b>1.093</b> | TLE           |
|             | $q_{80}$ | 4,483,887   |                              | TLE          | TLE          | <b>3.977</b>  |
|             | $q_{99}$ | 54,907      |                              | TLE          | 1.364        | <b>0.421</b>  |
| sc-ldoor    | $q_{20}$ | 274,585     | 1.679                        | TLE          | <b>4.870</b> | TLE           |
|             | $q_{80}$ | 13,473,317  |                              | TLE          | 563.996      | <b>14.944</b> |
|             | $q_{99}$ | 202,030     |                              | TLE          | 4.845        | <b>1.658</b>  |

Table 2: Median algorithm time in seconds on the fixed common grid. STRUCTBK is a structural-only reference independent of the quantile. Green marks the fastest completed ACMSC method; TLE denotes the 1,200-second limit.

## 6.5 Parallelism

Figure 7 compares the parallel behavior of both specialized algorithms. Dynamic edge-root scheduling gives MONOSEMMCE 5.62 $\times$  speedup at 32 threads on ogbn-arxiv  $q_{50}$ . For STRSUB, statistics collected separately from timing attribute 98.88% of the 32-thread BK-plus-local-processing work to intra-clique subset enumeration on FB15K-237  $q_{80}$ ; the final global filter takes 65 ms. We therefore partition sufficiently large fixed-size subset layers by their smallest selected vertex. Partitions within a layer are independent because equal-size sets cannot strictly contain one another, while a barrier between descending size layers preserves local-containment pruning. This bounded fine-grained schedule reaches 2.39 $\times$  speedup at 16 threads and 2.26 $\times$  at 32 threads without changing the output set or size histogram; the remaining saturation follows the uneven cost of host-clique subset tasks, not final containment filtering.

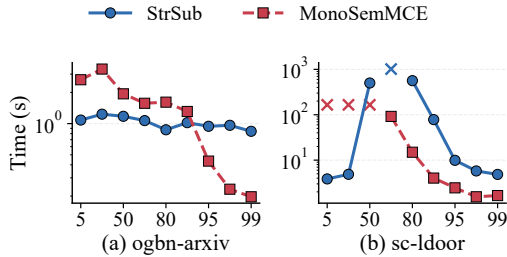


Figure 5: Runtime over the fixed nine-quantile grid. Crosses mark TLE; both methods use identical thresholds.

| Variant              | FB15K-237 $q_{80}$  | sc-nasasrb $q_{99}$ |
|----------------------|---------------------|---------------------|
|                      | Time (s) / peak GiB |                     |
| Full                 | 8.060 / 0.12        | 0.287 / 0.24        |
| No Top-Edge          | 7.462 / 0.12        | 0.324 / 0.24        |
| No vertex bound      | 5.898 / 0.12        | 0.312 / 0.24        |
| No local containment | 11.685 / 2.26       | 0.294 / 0.24        |

Table 3: Exact STRSUB component ablation. Each cell reports time in seconds / peak GiB; all variants return identical outputs.

## 6.6 Mechanisms and Exact Ablations

Figure 8 links observed runtimes to algorithm-specific work. On sc-nasasrb at  $q_{99}$ , the Top-Edge bound reduces subset tests from 16.50 million to 12.95 million, and the vertex-contribution bound reduces them from 80.28 million to 12.95 million. On ogbn-arxiv, increasing the threshold from  $q_5$  to  $q_{99}$  contracts accepted canonical states from 17.08 million to 18,011; at  $q_{99}$ , 10,272 terminal candidates are reduced to 10,261 final ACMSCs. The measurements explain why STRSUB remains stable across much of the grid and why MONOSEMMCE changes regime sharply near  $q_{95}$ .

Table 3 evaluates the three independently removable STRSUB components on two workloads. All four variants preserve 76,106 ACMSCs on FB15K-237 and 12,742 on sc-nasasrb. On sc-nasasrb, the complete design is the fastest; the Top-Edge and vertex-contribution bounds reduce subset tests from 16.50 and 80.28 million, respectively, to 12.95 million. On FB15K-237, local containment is the principal materialization control: disabling it increases peak memory from 0.12 to 2.26 GiB and runtime from 8.060 to 11.685 seconds.

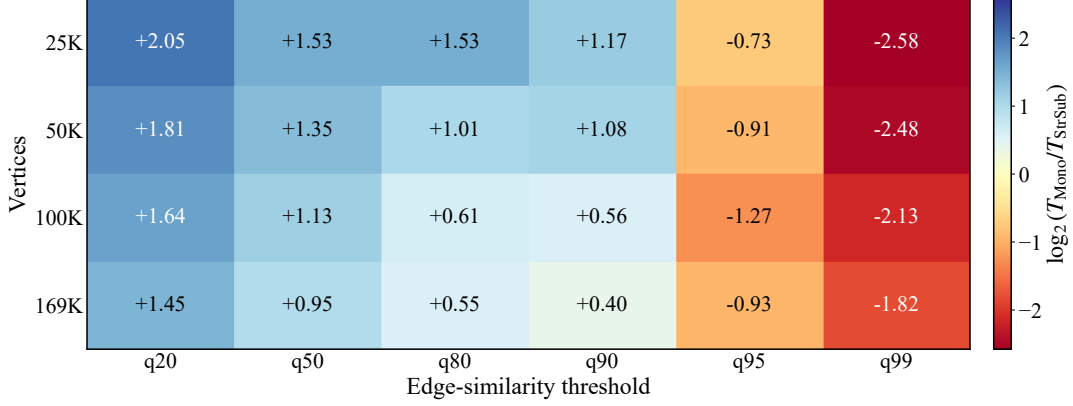


Figure 6: Runtime regime map on nested ogbn-arxiv induced subgraphs. Each cell is  $\log_2(T_{\text{Mono}}/T_{\text{StrSub}})$ ; blue favors STRSUB and red favors MONOSEMMCE.

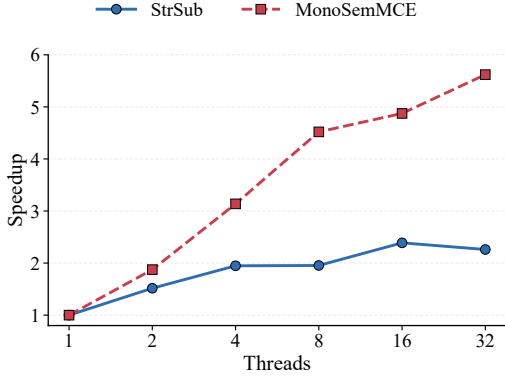


Figure 7: Speedup from 1 to 32 threads for one representative workload per specialized algorithm.

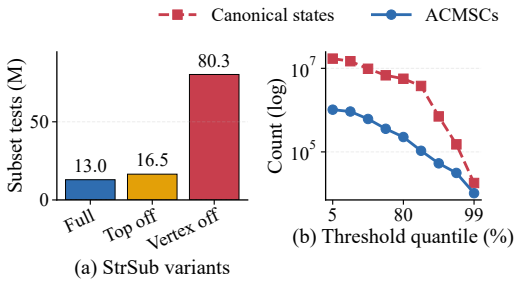


Figure 8: Search counters measured separately from runtime. Left: STRSUB bounds. Right: canonical states and ACMSC outputs.

Table 4 compares canonical reverse search with two alternative duplicate-control organizations that share the same similarity cache, feasibility test, and final containment filter. Canonical search is 6.27 $\times$  and 4.21 $\times$  faster than visited DFS and layered generation on ogbn-arxiv, and 4.21 $\times$  and 5.38 $\times$  faster on sc-nasasrb. It also has the

| Generation  | ogbn-arxiv $q_{95}$ |             | sc-nasasrb $q_{80}$ |             |
|-------------|---------------------|-------------|---------------------|-------------|
|             | Time (s)            | GiB         | Time (s)            | GiB         |
| Canonical   | <b>0.388</b>        | <b>0.39</b> | <b>0.729</b>        | <b>0.22</b> |
| Visited DFS | 2.432               | 0.53        | 3.069               | 0.64        |
| Layered     | 1.635               | 0.49        | 3.922               | 0.81        |

Table 4: MONOSEMMCE generation-strategy ablation with identical output sets.

smallest peak memory in both workloads. The three organizations return identical normalized clique sets and size histograms.

## 6.7 Output Space and Result Semantics

The final evaluation examines how the threshold changes the returned pattern space and whether the patterns align with external semantic labels. We first measure exact output distributions on FB15K-237, ogbn-arxiv, sc-nasasrb, and sc-lldoor over the fixed  $q_{20}$ ,  $q_{50}$ ,  $q_{80}$ ,  $q_{95}$ , and  $q_{99}$  grid. For each dataset, both the ACMSC count and mean size are normalized by the structural maximal cliques returned by STRUCTBK on the same graph.

Figure 9 compares the normalized transitions. The output-count ratios aggregated by the geometric mean are 1.04, 7.50, 3.53, 0.61, and 0.11. Intermediate thresholds split large structural cliques into multiple inclusion-maximal feasible subsets, while high thresholds contract the result space. The geometric-mean size ratio decreases from 1.06 at  $q_{20}$  to 0.39 at  $q_{99}$ . The individual curves retain the distinction between the two natural-semantic graphs and the two generated-attribute graphs, while the aggregate identifies the threshold-wide distribution shift.

The subject-label analysis uses ogbn-arxiv because its 40 official categories are independent of the mining objective. We additionally evaluate PAIRSIM-MCE, which retains an original edge when its end-point similarity reaches  $\tau$  and then enumerates structural maximal cliques. Subject purity is the largest official-category fraction in an output, averaged over non-singleton outputs. Figure 10 reports each threshold as a joint purity and vertex-coverage operating point;

the distribution analysis above supplies the corresponding change in pattern size. Across  $q_{50}$ – $q_{99}$ , ACMSC attains both higher mean purity and higher coverage than PAIRSIM-MCE. STRUCTBK provides the threshold-independent structural reference at full coverage.

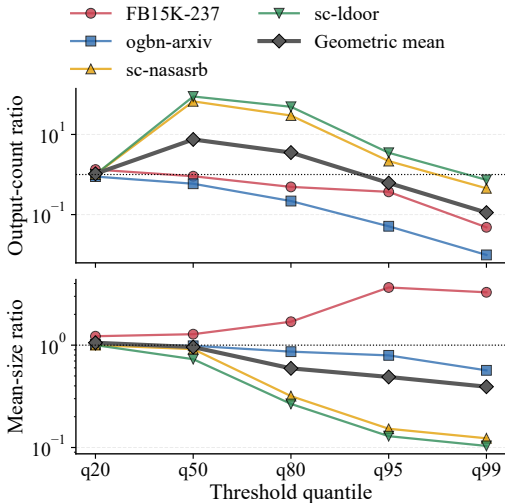
Figure 11 visualizes the largest label-pure ACMSC selected deterministically at  $q_{80}$ . The 18 papers all belong to cs.IT and study private information retrieval, spanning coded databases, cache-aided retrieval, private side information, wiretap channels, and private set intersection. Banawan, Ulukus, Wei, Jafar, and Sun recur across the group. Paper positions are initialized by classical multidimensional scaling of the cosine distances and relaxed only to separate overlapping nodes. Blue links form the union of each paper’s two most similar neighbors, red dashed links show all below-threshold pairs, and gold links connect authors appearing in at least three papers.

The group has average similarity 0.919 at  $\tau = 0.889$ , while 20 of its 153 pairs fall below the threshold and its minimum pair similarity is 0.821. The average constraint therefore retains the complete, collectively coherent group that the every-pair threshold divides. The case connects the aggregate label measurements to a concrete collaboration and research topic.

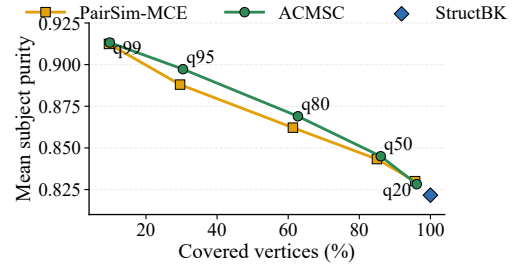
Across these experiments, STRSUB and MONOSEMMCE exhibit overlapping but distinct regions. STRSUB is preferred when structural MCE remains compact and semantic selectivity is low or intermediate. MONOSEMMCE is specialized for thresholds high enough to contract the feasible canonical tree, with the crossover appearing at  $q_{95}$  on the controlled scale series. The common grid, transition sweep, and controlled scale series connect this crossover to the two search organizations.

## 7 RELATED WORK

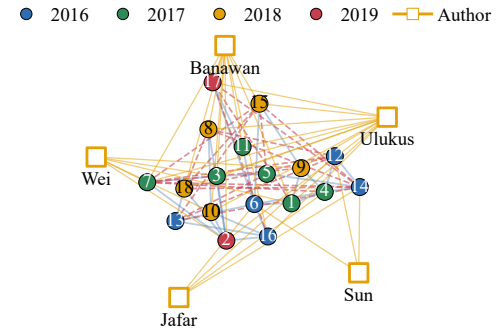
**Maximal Clique Enumeration.** The combinatorial output bound for maximal cliques was established by Moon and Moser [32], and



**Figure 9: Normalized output distributions on four datasets. Top: ACMSC count relative to STRUCTBK. Bottom: mean-size ratio. The thick line is the geometric mean.**



**Figure 10: Subject purity and vertex coverage on ogbn-arxiv over the fixed five-quantile grid. STRUCTBK is the threshold-independent structural reference.**



**Figure 11: A real 18-paper ACMSC on ogbn-arxiv. Paper positions follow cosine distance; blue links form the semantic backbone, red dashed links fall below  $q_{80}$ , and gold links show recurrent authors.**

polynomial-delay enumeration developed through maximal independent sets [30, 46] and cliques [5]. Bron–Kerbosch backtracking [4] became the standard practical skeleton, with Tomita pivoting attaining the worst-case output order [44]. Degeneracy-oriented variants exploit sparse real-world graphs [7, 13, 14]; out-of-core and parallel systems distribute the resulting search across massive graphs and multicore processors [8, 10, 27, 40]. These techniques justify the single STRUCTBK reference and the structural phase of STRSUB, but their pivot argument depends only on adjacency. It does not transfer to a global non-monotone average constraint.

**Attributed Subgraphs.** Classical cohesive-subgraph models trade structural strictness for scalable community objectives, including cores, modularity, and influence [6, 16, 29, 34, 41]. Attributed graph clustering combines topology with attribute distance or learned cluster objectives [3, 9, 37, 48, 49]. Pattern-mining formulations more closely connect structure and attributes: CoPaM enumerates dense connected subgraphs homogeneous in a feature subspace [33], GAMER joins subspace clustering with dense-subgraph mining [18], and structural-correlation patterns associate attribute sets with quasi-cliques [42]. Query-oriented systems instead return communities containing specified nodes or attributes under core or truss constraints [15, 21]. ACMSC differs at the contract level: it enumerates all structurally complete, inclusion-wise maximal sets under

one group-average vector constraint. Consequently, these methods motivate result quality comparisons, but are not interchangeable efficiency baselines.

**Vector Attributes and Similarity.** Vector-attributed graphs arise from knowledge-graph embeddings [2, 12, 35, 43], random-walk representations [17, 38], and message-passing models [19, 25]. Our natural workloads use text-derived sentence representations [39, 47] and the Open Graph Benchmark features [20], with FB15K-237 following the filtered knowledge-base split [45]. Approximate vector search accelerates unconstrained nearest-neighbor retrieval through hashing or navigable graphs [1, 22, 31], and GPU indexes extend this approach to billion-scale collections [23]. ACMSC needs similarities only on graph edges and must preserve an inclusive threshold exactly, so the implementation computes those edge similarities once and does not introduce approximate-neighbor error into enumeration.

**Similarity Search in Vector Spaces.** The precomputation of pairwise similarities leverages extensive work on vector similarity search [23]. We employ exact cosine similarity via AVX2 SIMD instructions because similarities are needed only for graph edges, rather than for all  $O(n^2)$  pairs. High-dimensional workloads could instead use locality-sensitive hashing [1] or graph-based nearest-neighbor indexes [31], although approximate integration is beyond the present work.

## 8 CONCLUSION

We introduced the average-constrained maximal semantic clique (ACMSC) problem, which identifies structurally complete cliques with guaranteed average pairwise semantic similarity. The problem generalizes classical MCE and is NP-complete. Its defining challenge is the non-monotonicity of the average-similarity constraint over the subset lattice, which invalidates the structural invariants that underpin the efficiency of standard Bron–Kerbosch algorithms.

We used SEMBK as a direct pivot-free baseline and proposed two exact algorithms with distinct operating regions. STRSUB decouples structural and semantic search, restores the Tomita pivot in a  $\tau$ -agnostic structural phase, and applies admissible edge and vertex-contribution bounds during exact subset enumeration. MONOSEM-MCE is a high- $\tau$  specialization: the deterministic deletion rule assigns each feasible clique one parent, so reverse DFS eliminates duplicate insertion paths without visited sets or materialized layers. Both methods use global strict-containment filtering to implement inclusion-wise maximality.

Exact-output tests validate the canonical search and every optional STRSUB component before timing. On ogbn-arxiv, STRSUB is fastest at the intermediate threshold, while MONOSEM-MCE is 4.28 $\times$  faster at  $q_{99}$ ; on FB15K-237, the corresponding high-threshold advantage is 4.48 $\times$ . Across the eight-dataset common grid, STRSUB completes 22 of 24 cells, and canonical search is 2.17 $\times$ –6.24 $\times$  faster on every  $q_{99}$  row. The controlled scale series places the crossover at  $q_{95}$  across four graph sizes, while canonical edge-root scheduling reaches 5.62 $\times$  speedup with 32 threads. Mechanism counters connect these results to exact layer bounds in STRSUB and the rapid high-threshold contraction of the canonical tree in MONOSEM-MCE. The fixed five-threshold output study also captures the intermediate pattern-count expansion and high-threshold contraction, while

external labels and a real paper group establish the semantic content of the returned ACMSCs. Future directions include stronger admissible multi-step bounds, finer canonical-subtree scheduling, and dynamic-graph extensions.

## REFERENCES

- [1] Alexandr Andoni and Piotr Indyk. 2006. Near-Optimal Hashing Algorithms for Approximate Nearest Neighbor in High Dimensions. In *IEEE Symposium on Foundations of Computer Science*. IEEE, 459–468.
- [2] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. 2013. Translating Embeddings for Modeling Multi-Relational Data. In *Advances in Neural Information Processing Systems*. 2787–2795.
- [3] Cécile Bothorel, Juan David Cruz, Matteo Magnani, and Barbora Mícenková. 2015. Clustering Attributed Graphs: Models, Measures and Methods. *Network Science* (2015), 408–444.
- [4] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: Finding All Cliques of an Undirected Graph. *Commun. ACM* (1973), 575–577.
- [5] Frédéric Cazals and Chinmay Karande. 2008. A Note on the Problem of Reporting Maximal Cliques. *Theoretical Computer Science* (2008), 564–579.
- [6] Lijun Chang and Lu Qin. 2018. *Cohesive Subgraph Computation over Large Sparse Graphs*. Springer.
- [7] Lijun Chang, Jeffrey Xu Yu, and Lu Qin. 2013. Fast Maximal Cliques Enumeration in Sparse Graphs. *Algorithmica* (2013), 173–186.
- [8] James Cheng, Yiping Ke, Ada Wai-Chee Fu, Jeffrey Xu Yu, and Linhong Zhu. 2011. Finding Maximal Cliques in Massive Networks by H\*-Graph. *ACM Transactions on Database Systems* (2011), 21:1–21:34.
- [9] Petr Chumae. 2020. Community Detection in Node-Attributed Social Networks: A Survey. *Computer Science Review* (2020), 100286.
- [10] Apurba Das, Michael D. Shah, Shreyas Pai, and Sriram V. Pemmaraju. 2019. A Shared-Memory Parallel Algorithm for Maximal Clique Enumeration from Large Sparse Graphs. In *IEEE International Parallel and Distributed Processing Symposium*. IEEE, 461–472.
- [11] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Software* (2011), 1:1–1:25.
- [12] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D Knowledge Graph Embeddings. In *AAAI Conference on Artificial Intelligence*. AAAI Press, 1811–1818.
- [13] David Eppstein, Maarten Löffler, and Darren Strash. 2010. Listing All Maximal Cliques in Sparse Graphs in Near-Optimal Time. In *International Symposium on Algorithms and Computation*. Springer, 403–414.
- [14] David Eppstein and Darren Strash. 2013. Listing All Maximal Cliques in Large Sparse Real-World Graphs. *ACM Journal of Experimental Algorithmics* (2013), 3:1:1–3:1:21.
- [15] Yixiang Fang, Reynold Cheng, Xiaodong Li, and Siqiang Luo. 2016. Effective Community Search for Large Attributed Graphs. *Proceedings of the VLDB Endowment* (2016), 1233–1244.
- [16] Santo Fortunato. 2010. Community Detection in Graphs. *Physics Reports* (2010), 75–174.
- [17] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 855–864.
- [18] Stephan Günnemann, Ines Färber, Brigitte Boden, and Thomas Seidl. 2010. Subspace Clustering Meets Dense Subgraph Mining: A Synthesis of Two Paradigms. In *IEEE International Conference on Data Mining*. IEEE, 845–850.
- [19] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*. 1024–1034.
- [20] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *Advances in Neural Information Processing Systems*. 22118–22133.
- [21] Xin Huang and Laks V. S. Lakshmanan. 2017. Attribute-Driven Community Search. *Proceedings of the VLDB Endowment* (2017), 949–960.
- [22] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *ACM Symposium on Theory of Computing*. ACM, 604–613.
- [23] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* (2021), 535–547.
- [24] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*. Plenum Press, 85–103.
- [25] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [26] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection.

- [27] Byron Lessley, Talita Perciano, and Hank Childs. 2016. Data-Parallel Techniques for High-Performance Maximal Clique Enumeration. In *IEEE International Conference on Data Science and Advanced Analytics*. IEEE, 636–645.
- [28] Rong-Hua Li, Qiangqiang Dai, Lu Qin, Guoren Wang, Xiaokui Xiao, and Jeffrey Xu Yu. 2018. Efficient Signed Clique Search in Signed Networks. In *34th IEEE International Conference on Data Engineering*. IEEE.
- [29] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2015. Influential Community Search in Large Networks. *Proceedings of the VLDB Endowment* (2015), 509–520.
- [30] Kazuhisa Makino and Takeaki Uno. 2004. New Algorithms for Enumerating All Maximal Cliques. In *Scandinavian Workshop on Algorithm Theory*. Springer, 260–272.
- [31] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2020), 824–836.
- [32] John W. Moon and Leo Moser. 1965. On Cliques in Graphs. *Israel Journal of Mathematics* (1965), 23–28.
- [33] Flavia Moser, Recep Colak, Arash Rafiey, and Martin Ester. 2009. Mining Cohesive Patterns from Graphs with Feature Vectors. In *SIAM International Conference on Data Mining*. SIAM, 593–604.
- [34] M. E. J. Newman. 2006. Modularity and Community Structure in Networks. *Proceedings of the National Academy of Sciences* (2006), 8577–8582.
- [35] Maximilian Nickel, Volker Tresp, and Hans-Peter Kriegel. 2011. A Three-Way Model for Collective Learning on Multi-Relational Data. In *International Conference on Machine Learning*. Omnipress, 809–816.
- [36] Marie Pelleau, Laurent Simon, and Jean-Charles Régin. 2025. Enumerating Cliques of Hypergraphs. In *37th IEEE International Conference on Tools with Artificial Intelligence*. IEEE.
- [37] Bryan Perozzi, Leman Akoglu, and Patricia Iglesias Sánchez. 2014. Focused Clustering and Outlier Detection in Large Attributed Graphs. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 1346–1355.
- [38] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. DeepWalk: Online Learning of Social Representations. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 701–710.
- [39] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 3982–3992.
- [40] Ryan A. Rossi, Rong Zhou, and Nesreen K. Ahmed. 2015. Parallel Maximum Clique Algorithms with Applications to Network Analysis and Storage. *SIAM Journal on Scientific Computing* (2015), C589–C616.
- [41] Stephen B. Seidman. 1983. Network Structure and Minimum Degree. *Social Networks* (1983), 269–287.
- [42] Arlei Silva, Wagner Meira Jr., and Mohammed J. Zaki. 2012. Mining Attribute-Structure Correlated Patterns in Large Attributed Graphs. *Proceedings of the VLDB Endowment* (2012), 466–477.
- [43] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. 2019. RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space. In *International Conference on Learning Representations*.
- [44] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The Worst-Case Time Complexity for Generating All Maximal Cliques and Computational Experiments. *Theoretical Computer Science* (2006), 28–42.
- [45] Kristina Toutanova and Danqi Chen. 2015. Observed versus Latent Features for Knowledge Base and Text Inference. In *Workshop on Continuous Vector Space Models and their Compositionality*. Association for Computational Linguistics, 57–66.
- [46] Shuichi Tsukiyama, Mikio Ide, Hiromu Ariyoshi, and Isao Shirakawa. 1977. A New Algorithm for Generating All the Maximal Independent Sets. *SIAM J. Comput.* (1977), 505–517.
- [47] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*. 5776–5788.
- [48] Jaewon Yang, Julian McAuley, and Jure Leskovec. 2013. Community Detection in Networks with Node Attributes. In *IEEE International Conference on Data Mining*. IEEE, 1151–1156.
- [49] Yang Zhou, Hong Cheng, and Jeffrey Xu Yu. 2009. Graph Clustering Based on Structural/Attribute Similarities. In *Proceedings of the VLDB Endowment*. 718–729.